

let's now go deep into Doubly Linked List (DLL) in Java, with:

- 💡 Complete concepts
- ⚙️ All operations (insert, delete, traverse, search)
- 📄 Key notes & important recall points
- 💻 Example code

Perfect 👍 Let's go **deep into Doubly Linked List (DLL) in Java** — covering *all concepts, operations, important notes, and example code* for complete understanding.

---

## 💡 Concept of Doubly Linked List (DLL)

A **Doubly Linked List** is a type of **Linked List** in which each node has **three parts**:

1. **Data** – stores the actual information.
2. **Prev** – reference to the previous node.
3. **Next** – reference to the next node.

Unlike a **Singly Linked List**, a DLL allows **traversal in both directions** (forward and backward).

---

## 🗨️ Structure of a Doubly Linked List Node

java

📋 Copy code

```
class Node {  
    int data;  
    Node prev;  
    Node next;  
  
    Node(int data) {  
        this.data = data;  
        this.prev = null;  
        this.next = null;  
    }  
}
```

```
}  
}
```

## ⚙️ Core Operations in DLL

We'll go through all important operations one by one with example code snippets.

### 1 Insertion Operations

#### a. Insert at the Beginning

- Create a new node.
- Set its `next` to the current head.
- Update the previous head's `prev` to the new node.
- Move `head` to the new node.

java

 Copy code

```
void insertAtBeginning(int data) {  
    Node newNode = new Node(data);  
    if (head == null) {  
        head = newNode;  
        return;  
    }  
    newNode.next = head;  
    head.prev = newNode;  
    head = newNode;  
}
```

#### b. Insert at the End

- Traverse to the last node.
- Set the new node's `prev` to last node and last node's `next` to new node.

java

 Copy code

```
void insertAtEnd(int data) {  
    Node newNode = new Node(data);  
    if (head == null) {  
        head = newNode;  
    }
```

```
        return;
    }
    Node temp = head;
    while (temp.next != null)
        temp = temp.next;
    temp.next = newNode;
    newNode.prev = temp;
}
```

---

### c. Insert After a Given Node

java

 Copy code

```
void insertAfter(Node prevNode, int data) {
    if (prevNode == null) {
        System.out.println("Previous node cannot be null");
        return;
    }
    Node newNode = new Node(data);
    newNode.next = prevNode.next;
    prevNode.next = newNode;
    newNode.prev = prevNode;
    if (newNode.next != null)
        newNode.next.prev = newNode;
}
```

---

## 2 Deletion Operations

### a. Delete from Beginning

java

 Copy code

```
void deleteFromBeginning() {
    if (head == null) return;
    if (head.next == null) {
        head = null;
        return;
    }
    head = head.next;
```

```
    head.prev = null;
}
```

## b. Delete from End

java

 Copy code

```
void deleteFromEnd() {
    if (head == null) return;
    if (head.next == null) {
        head = null;
        return;
    }
    Node temp = head;
    while (temp.next != null)
        temp = temp.next;
    temp.prev.next = null;
}
```

## c. Delete by Value

java

 Copy code

```
void deleteByValue(int key) {
    if (head == null) return;

    Node temp = head;

    // if head itself holds the key
    if (temp.data == key) {
        head = temp.next;
        if (head != null)
            head.prev = null;
        return;
    }

    // search for node to delete
    while (temp != null && temp.data != key)
        temp = temp.next;
```

```
        if (temp == null) return;

        if (temp.next != null)
            temp.next.prev = temp.prev;

        if (temp.prev != null)
            temp.prev.next = temp.next;
    }
}
```

### 3 Traversal Operations

#### a. Forward Traversal

java

 Copy code

```
void traverseForward() {
    Node temp = head;
    System.out.print("Forward: ");
    while (temp != null) {
        System.out.print(temp.data + " ");
        temp = temp.next;
    }
    System.out.println();
}
```

#### b. Backward Traversal

java

 Copy code

```
void traverseBackward() {
    Node temp = head;
    if (temp == null) return;
    while (temp.next != null)
        temp = temp.next;
    System.out.print("Backward: ");
    while (temp != null) {
        System.out.print(temp.data + " ");
        temp = temp.prev;
    }
    System.out.println();
}
```

## Search Operation

java

 Copy code

```
boolean search(int key) {
    Node temp = head;
    while (temp != null) {
        if (temp.data == key)
            return true;
        temp = temp.next;
    }
    return false;
}
```



## Complete Example Code

java

 Copy code

```
class Node {
    int data;
    Node prev, next;

    Node(int data) {
        this.data = data;
        this.prev = null;
        this.next = null;
    }
}

public class DoublyLinkedList {
    Node head;

    void insertAtBeginning(int data) {
        Node newNode = new Node(data);
        if (head == null) {
            head = newNode;
            return;
        }
        newNode.next = head;
        head.prev = newNode;
        head = newNode;
    }
}
```

```
}

void insertAtEnd(int data) {
    Node newNode = new Node(data);
    if (head == null) {
        head = newNode;
        return;
    }
    Node temp = head;
    while (temp.next != null)
        temp = temp.next;
    temp.next = newNode;
    newNode.prev = temp;
}

void deleteFromBeginning() {
    if (head == null) return;
    if (head.next == null) {
        head = null;
        return;
    }
    head = head.next;
    head.prev = null;
}

void deleteFromEnd() {
    if (head == null) return;
    if (head.next == null) {
        head = null;
        return;
    }
    Node temp = head;
    while (temp.next != null)
        temp = temp.next;
    temp.prev.next = null;
}

void traverseForward() {
    Node temp = head;
    System.out.print("Forward: ");
    while (temp != null) {
        System.out.print(temp.data + " ");
        temp = temp.next;
    }
}
```

```
    }
    System.out.println();
}

void traverseBackward() {
    Node temp = head;
    if (temp == null) return;
    while (temp.next != null)
        temp = temp.next;
    System.out.print("Backward: ");
    while (temp != null) {
        System.out.print(temp.data + " ");
        temp = temp.prev;
    }
    System.out.println();
}

boolean search(int key) {
    Node temp = head;
    while (temp != null) {
        if (temp.data == key)
            return true;
        temp = temp.next;
    }
    return false;
}

public static void main(String[] args) {
    DoublyLinkedList dll = new DoublyLinkedList();

    dll.insertAtBeginning(10);
    dll.insertAtEnd(20);
    dll.insertAtEnd(30);
    dll.insertAtBeginning(5);

    dll.traverseForward();
    dll.traverseBackward();

    System.out.println("Search 20: " + dll.search(20));
    System.out.println("Search 99: " + dll.search(99));

    dll.deleteFromBeginning();
    dll.deleteFromEnd();
}
```



```
        dll.traverseForward();  
    }  
}
```

---

## Key Notes & Recall Points

- ✓ DLL allows **bidirectional traversal** using `prev` and `next` .
- ✓ More **memory** required per node (extra `prev` pointer).
- ✓ **Insertion and deletion** are easier compared to Singly Linked List (no need to track previous node manually).
- ✓ **Head node's prev** is always `null` , and **last node's next** is `null` .
- ✓ Common uses:
  - Undo/Redo functionality.
  - Browser history navigation.
  - LRU (Least Recently Used) Cache implementation.

---

Would you like me to continue next with **Circular Doubly Linked List (CDLL)** — its working and operations (insert, delete, traverse, etc.)?

